

Thẻ 1

CÁC CHỦ ĐỀ ĐƯỢC ĐỀ CẬP (TOPICS COVERED)

- **Agile methods (Phương pháp linh hoạt):** Các phương pháp luận tiếp cận việc phát triển phần mềm theo hướng linh hoạt.
 - **Agile development techniques (Kỹ thuật phát triển linh hoạt):** Các kỹ thuật cụ thể được áp dụng trong quy trình Agile.
 - **Agile project management (Quản trị dự án linh hoạt):** Cách thức điều hành và quản lý dự án theo tư duy Agile.
-

PHÁT TRIỂN PHẦN MỀM NHANH (RAPID SOFTWARE DEVELOPMENT)

- **Rapid development and delivery (Phát triển và bàn giao nhanh)** hiện nay thường là **requirement (yêu cầu)** quan trọng nhất đối với các hệ thống phần mềm.
- Các doanh nghiệp hoạt động trong một môi trường **fast-changing requirement (yêu cầu thay đổi nhanh chóng)**.

-> Gần như không thể có được các **stable software requirements (yêu cầu phần mềm ổn định)**.

- Phần mềm phải **evolve (tiến hóa/phát triển dần)** nhanh chóng để phản ánh các nhu cầu kinh doanh thay đổi.
 - **Plan-driven development (Phát triển dựa trên kế hoạch)** (ví dụ: **waterfall - mô hình thác nước, incremental dev - phát triển tăng trưởng**) là thiết yếu cho một số loại hệ thống nhưng không đáp ứng được các nhu cầu kinh doanh này.
-

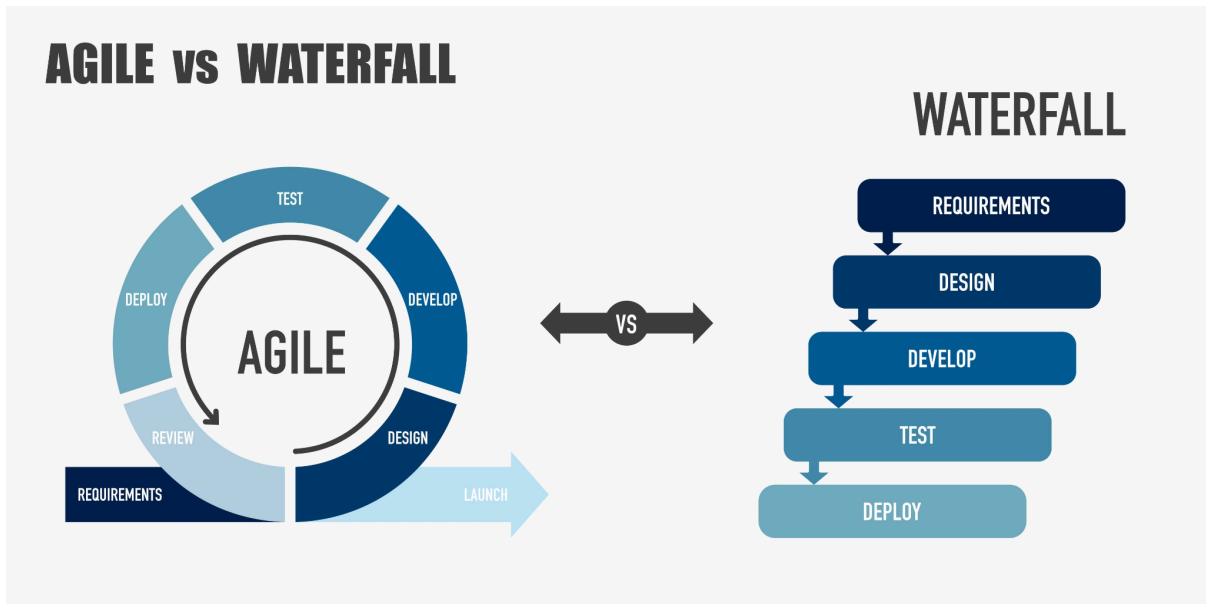
PHÁT TRIỂN LINH HOẠT (AGILE DEVELOPMENT)

Xuất hiện vào cuối những năm 1990.

*Mục tiêu nhằm giảm thiểu đáng kể **delivery time (thời gian bàn giao)** cho các hệ thống phần mềm đang hoạt động (**working software systems**).*

- **Program specification (Đặc tả chương trình), design (thiết kế) và implementation (triển khai/cài đặt)** được thực hiện **inter-leaved (đan xen với nhau)**.
 - Hệ thống được coi là một chuỗi các **versions (phiên bản)** hoặc **increments (phần tăng trưởng)**.
 - **Stakeholders (Các bên liên quan)** tham gia vào việc đặc tả và **evaluation (đánh giá)** phiên bản.

- Thường xuyên bàn giao các phiên bản mới để đánh giá.
- **Minimal documentation (Tài liệu hóa tối giản)** – tập trung vào **working code (mã nguồn đang chạy tốt)**.
- Sử dụng sự hỗ trợ rộng rãi từ công cụ (ví dụ: **automated testing tools - công cụ kiểm thử tự động**) để hỗ trợ quá trình phát triển.



💡 MỞ RỘNG (EXPANSION)

1. Sự khác biệt cốt lõi (Agile vs. Plan-driven):

- Trong **Plan-driven**, các giai đoạn được tách biệt rõ ràng: hoàn thành xong Đặc tả mới chuyển sang Thiết kế.
- Trong **Agile**, ranh giới này bị xóa bỏ. Bạn vừa làm đặc tả, vừa code, vừa test liên tục.

2. Tại sao lại là "Minimal Documentation"?:

- Trong môi trường thay đổi nhanh, tài liệu dày cộp sẽ nhanh chóng trở nên lạc hậu (**outdated**). Thay vì mất thời gian cập nhật tài liệu, Agile ưu tiên việc viết code sạch, dễ hiểu và có các bộ **Unit Test** tự động để chứng minh phần mềm chạy đúng.

3. Tuyên ngôn Agile (Agile Manifesto): Để hiểu sâu hơn, bạn nên nhớ 4 giá trị cốt lõi của Agile thường xuất hiện trong câu hỏi trắc nghiệm:

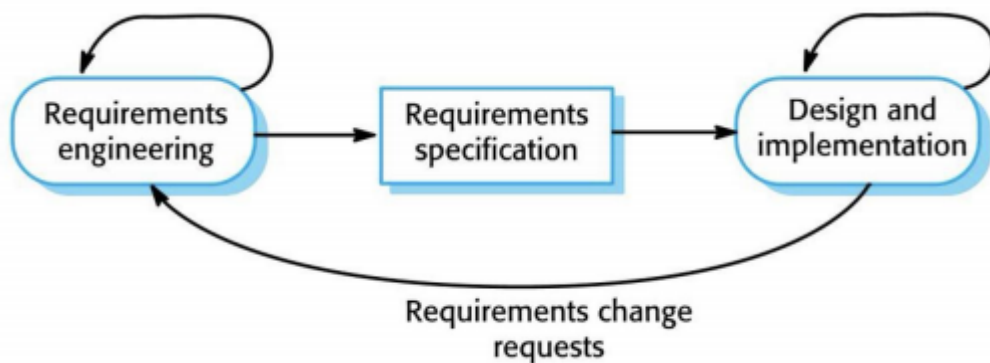
- **Individuals and interactions (Cá nhân và sự tương tác)** hơn là quy trình và công cụ.
- **Working software (Phần mềm chạy tốt)** hơn là tài liệu đầy đủ.
- **Customer collaboration (Cộng tác với khách hàng)** hơn là đàm phán hợp đồng.

- **Responding to change (Phản hồi với thay đổi)** hơn là bám sát kế hoạch.
 - 4. **Stakeholders (Bên liên quan):** Khái niệm này không chỉ là khách hàng, mà bao gồm tất cả những người có ảnh hưởng hoặc bị ảnh hưởng bởi dự án (người dùng, quản lý, lập trình viên, nhà đầu tư). Trong Agile, sự hiện diện của họ ở mỗi đợt bàn giao phiên bản là cực kỳ quan trọng để đảm bảo đội ngũ đang đi đúng hướng.
-

PHÁT TRIỂN DỰA TRÊN KẾ HOẠCH VÀ PHÁT TRIỂN LINH HOẠT (PLAN-DRIVEN AND AGILE DEVELOPMENT)

1. Phát triển dựa trên kế hoạch (Plan-based development)

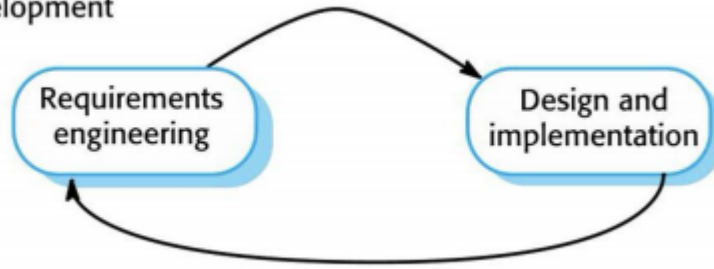
Ví dụ: Mô hình thác nước (Waterfall model), Phát triển tăng dần (Incremental development)



- **Kỹ nghệ yêu cầu (Requirements engineering):** Quá trình tìm hiểu, phân tích và xác định các dịch vụ mà hệ thống phải cung cấp cũng như các ràng buộc của nó.
- **Đặc tả yêu cầu (Requirements specification):** Tài liệu chính thức ghi lại các yêu cầu đã được thống nhất, đóng vai trò là "bản hợp đồng" giữa khách hàng và nhóm phát triển.
- **Thiết kế và triển khai (Design and implementation):** Giai đoạn chuyển hóa các đặc tả thành cấu trúc phần mềm và viết mã nguồn thực tế.
- **Các yêu cầu thay đổi yêu cầu (Requirements change requests):** Khi có sự thay đổi trong quá trình thực hiện, quy trình phải quay ngược lại từ giai đoạn triển khai về giai đoạn kỹ nghệ yêu cầu để điều chỉnh lại từ đầu.

2. Phát triển linh hoạt (Agile development)

Agile development



- Quy trình này lược bỏ bước tạo ra một bản **Đặc tả yêu cầu (Requirements specification)** trung gian và cứng nhắc.
- **Vòng lặp liên tục:** Hoạt động **Kỹ nghệ yêu cầu (Requirements engineering)** và **Thiết kế và triển khai (Design and implementation)** được thực hiện song hành và lặp đi lặp lại.
- Thay đổi được chấp nhận và tích hợp liên tục vào các vòng lặp phát triển thay vì phải thông qua các quy trình phê duyệt thay đổi phức tạp.

💡 MỞ RỘNG (EXPANSION)

- **Sự khác biệt về Tài liệu (Documentation):** * Trong **Phát triển dựa trên kế hoạch (Plan-driven)**, bản **Đặc tả yêu cầu (Requirements specification)** là cực kỳ quan trọng vì nó là cơ sở để lập kế hoạch chi tiết, ước tính ngân sách và thời gian.
 - Trong **Phát triển linh hoạt (Agile)**, tài liệu này thường được thay thế bằng các **Câu chuyện người dùng (User Stories)** ngắn gọn và được cập nhật liên tục trong các **Kỳ chạy nước rút (Sprints)**.
 - **Phản ứng với thay đổi:** * Sơ đồ cho thấy trong mô hình truyền thống, khi có **Yêu cầu thay đổi (Change requests)**, chi phí sửa đổi thường rất cao vì phải cập nhật lại toàn bộ hệ thống tài liệu và thiết kế đã hoàn thiện trước đó.
 - Ngược lại, Agile coi thay đổi là một phần tự nhiên của quy trình, giúp giảm thiểu rủi ro xây dựng sai sản phẩm ngay từ đầu.
 - **Khi nào chọn phương pháp nào?**
 - **Plan-driven:** Phù hợp với các hệ thống yêu cầu độ an toàn cao (Safety-critical systems), các dự án có đội ngũ phát triển phân tán ở nhiều quốc gia, hoặc các dự án có yêu cầu rất rõ ràng ngay từ đầu.
 - **Agile:** Phù hợp với các sản phẩm thương mại, ứng dụng di động, hoặc các dự án startup nơi yêu cầu của thị trường thay đổi từng ngày.
-

CÁC PHƯƠNG PHÁP LINH HOẠT (AGILE METHODS)

- **Các phương pháp linh hoạt (Agile methods):**
 - Tập trung vào **Mã nguồn (Code)** thay vì **Thiết kế (Design)**.
 - Dựa trên **Tiếp cận lặp (Iterative approach)** đối với **Phát triển phần mềm (Software development)**.
 - Mục đích là để bàn giao **Phần mềm đang chạy tốt (Working software)** một cách nhanh chóng và **Tiến hóa (Evolve)** nó nhanh chóng để đáp ứng **Các yêu cầu thay đổi (Changing requirements)**.
 - **Mục tiêu (Aims):**
 - **Giảm thiểu các Chi phí gián tiếp/Chi phí dư thừa (Overheads)** trong **Quy trình phần mềm (Software process)** (ví dụ: bằng cách hạn chế **Tài liệu hóa (Documentation)**).
 - **Phản hồi nhanh chóng với các Yêu cầu thay đổi (Changing requirements)** mà không cần **Làm lại quá mức (Excessive rework)**.
-

MỞ RỘNG (EXPANSION)

1. **Tập trung vào Mã nguồn thay vì Thiết kế (Focus on code vs. design):**
 - Điều này **không** có nghĩa là Agile bỏ qua việc thiết kế. Thay vào đó, nó phản đối tư duy **Thiết kế lớn ngay từ đầu (Big Design Up Front - BDUF)** của mô hình truyền thống. Trong Agile, thiết kế được thực hiện song song và tiến hóa dần cùng với mã nguồn (được gọi là **Thiết kế tiến hóa - Evolutionary Design**).
2. **Chi phí gián tiếp (Overheads) trong quy trình:**
 - Trong mô hình Thác nước, các cuộc họp phê duyệt, việc viết các bản đặc tả hàng trăm trang và quy trình kiểm soát thay đổi khắt khe được coi là "overheads" vì chúng không trực tiếp tạo ra giá trị phần mềm cho khách hàng. Agile cố gắng cắt tía những thứ này để tập trung nguồn lực vào việc viết mã tạo ra giá trị.
3. **Tiếp cận lặp (Iterative approach):**
 - Thay vì bàn giao một sản phẩm hoàn chỉnh duy nhất vào cuối dự án (thường mất nhiều tháng hoặc năm), Agile chia dự án thành các vòng lặp ngắn (thường từ 1-4 tuần). Mỗi vòng lặp kết thúc bằng một phiên bản phần mềm có thể chạy được và mang lại giá trị nhất định.
4. **Hạn chế làm lại quá mức (Excessive rework):**
 - Bằng cách làm việc theo các chu kỳ ngắn và liên tục nhận phản hồi từ khách hàng, đội ngũ phát triển phát hiện ra sai sót hoặc sự hiểu lầm về

yêu cầu ngay lập tức. Điều này giúp tránh tình trạng đến cuối dự án mới nhận ra sai lầm và phải đập đi xây lại toàn bộ hệ thống.

5. Tại sao lại cần tiến hóa nhanh (Evolve quickly)?

- Trong thị trường hiện đại, một tính năng hôm nay là đột phá nhưng ngày mai có thể trở thành lỗi thời. Khả năng "tiến hóa" của phần mềm giúp doanh nghiệp thích nghi với các cơ hội thị trường mới hoặc các quy định pháp lý thay đổi bất ngờ

CÁC NGUYÊN TẮC CỦA PHƯƠNG PHÁP LINH HOẠT (THE PRINCIPLES OF AGILE METHODS)

Nguyên tắc (Principle)	Mô tả (Description)
Sự tham gia của khách hàng (Customer involvement)	Khách hàng nên tham gia chặt chẽ trong suốt Quy trình phát triển (Development process) . Vai trò của họ là cung cấp và ưu tiên các Yêu cầu hệ thống mới (New system requirements) và đánh giá các Vòng lặp (Iterations) của hệ thống.
Bàn giao tăng dần (Incremental delivery)	Phần mềm được phát triển theo từng Phần tăng trưởng (Increments) , trong đó khách hàng sẽ chỉ định các Yêu cầu (Requirements) được đưa vào trong mỗi phần tăng trưởng đó.
Con người, không phải quy trình (People, not process)	Các kỹ năng của Đội ngũ phát triển (Development team) cần được công nhận và khai thác. Các thành viên trong nhóm nên được để tự phát triển cách làm việc riêng của họ mà không cần các Quy trình mang tính bắt buộc (Prescriptive processes) .

Chấp nhận thay đổi (Embrace change)	Luôn dự đoán rằng các Yêu cầu hệ thống (System requirements) sẽ thay đổi, do đó cần thiết kế hệ thống để thích ứng với những thay đổi này.
Duy trì sự đơn giản (Maintain simplicity)	Tập trung vào sự đơn giản trong cả phần mềm đang được phát triển và trong Quy trình phát triển (Development process) . Bất cứ khi nào có thể, hãy chủ động làm việc để loại bỏ Sự phức tạp (Complexity) khỏi hệ thống.

KHẢ NĂNG ÁP DỤNG PHƯƠNG PHÁP LINH HOẠT (AGILE METHOD APPLICABILITY)

- Dành cho các **Sản phẩm thương mại (Product for sale)** quy mô nhỏ hoặc trung bình.
 - Hầu như tất cả các sản phẩm phần mềm và ứng dụng hiện nay đều được phát triển bằng **Tiếp cận linh hoạt (Agile approach)**.
- Có **Sự cam kết rõ ràng từ khách hàng (Clear commitment from the customer)** để tham gia sâu:
 - Vào **Quy trình phát triển (Development process)**.
 - Và ở những nơi có ít **Quy tắc và quy định bên ngoài (External rules and regulations)** tác động đến phần mềm.

MỞ RỘNG (EXPANSION)

1. **Vai trò của Khách hàng:** Trong Agile, khách hàng không chỉ đứng ngoài quan sát mà thường **đóng vai trò là một thành viên của đội dự án (trong Scrum gọi là Product Owner)**. Nếu khách hàng không thể tham gia thường xuyên, dự án Agile rất dễ thất bại vì đội ngũ sẽ thiếu đi sự chỉ dẫn về độ ưu tiên của tính năng.
2. **Nguyên tắc YAGNI (You Ain't Gonna Need It):** Đây là một triết lý đi kèm với nguyên tắc **Duy trì sự đơn giản**. Lập trình viên chỉ nên viết mã cho các chức năng hiện tại đang cần, thay vì đoán trước tương lai và làm cho hệ thống phức tạp một cách không cần thiết.
3. **Hạn chế của Agile:** Slide có đề cập đến "external rules and regulations". Điều này giải thích tại sao các hệ thống như phần mềm điều khiển máy bay, thiết bị y tế hoặc hạt nhân thường khó áp dụng hoàn toàn Agile. Những ngành này yêu

cầu **Tài liệu hóa (Documentation)** cực kỳ chi tiết và khắt khe để đảm bảo an toàn, điều mà Agile thường lược bỏ để tăng tốc độ.

4. **Đội ngũ tự quản (Self-organizing teams):** Nguyên tắc **Con người, không phải quy trình** nhấn mạnh rằng các lập trình viên giỏi sẽ biết cách tốt nhất để giải quyết vấn đề của họ. Thay vì áp đặt một quy trình cứng nhắc từ trên xuống, nhà quản lý đóng vai trò hỗ trợ để đội ngũ tự vận hành.

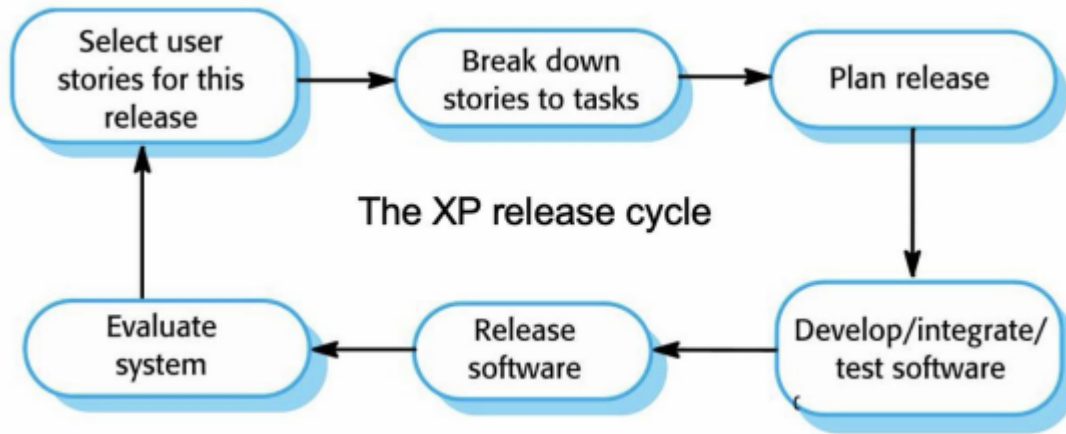
SO SÁNH QUY TRÌNH PHÁT TRIỂN (DEVELOPMENT PROCESS COMPARISON)

- **Phát triển dựa trên kế hoạch (Plan-based development):** Ví dụ như **Mô hình thác nước (Waterfall model)** hoặc **Phát triển tăng dần (Incremental development)**.
 - Quy trình đi từ **Kỹ nghệ yêu cầu (Requirements engineering)** sang **Đặc tả yêu cầu (Requirements specification)**, sau đó mới đến **Thiết kế và triển khai (Design and implementation)**.
 - Khi có **Yêu cầu thay đổi yêu cầu (Requirements change requests)**, quy trình phải quay ngược lại giai đoạn đầu.
- **Phát triển linh hoạt (Agile development):** Các hoạt động **Kỹ nghệ yêu cầu (Requirements engineering)** và **Thiết kế và triển khai (Design and implementation)** được thực hiện đan xen và lặp lại liên tục.

LẬP TRÌNH CỰC HẠN (EXTREME PROGRAMMING - XP)

- **Lập trình cực hạn (Extreme Programming - XP)** áp dụng một cách tiếp cận "cực hạn" đối với **Phát triển lặp (Iterative development)**.
- Các **Phiên bản mới (New versions)** có thể được xây dựng nhiều lần trong ngày.
- Các **Phần tăng trưởng (Increments)** được bàn giao cho khách hàng mỗi 2 tuần.
- Tất cả các **Kiểm thử (Tests)** phải được thực hiện cho mọi **Bản dựng (Build)** và bản dựng chỉ được chấp nhận nếu các kiểm thử chạy thành công.

Chu trình phát hành của XP (The XP release cycle)



Quy trình vận hành theo vòng lặp:

1. Chọn câu chuyện người dùng cho đợt phát hành này (Select user stories for this release).
2. Chia nhỏ các câu chuyện thành các nhiệm vụ (Break down stories to tasks).
3. Lập kế hoạch phát hành (Plan release).
4. Phát triển/Tích hợp/Kiểm thử phần mềm (Develop/integrate/test software).
5. Phát hành phần mềm (Release software).
6. Đánh giá hệ thống (Evaluate system) và quay lại bước 1.

CÁC THỰC TIỄN TRONG LẬP TRÌNH CỰC HẠN (EXTREME PROGRAMMING PRACTICES)

Nguyên tắc hoặc Thực tiễn (Principle or practice)	Mô tả (Description)
Lập kế hoạch tăng dần (Incremental planning)	Các yêu cầu được ghi lại trên Thẻ câu chuyện (Story cards) ; việc chọn câu chuyện nào để phát hành dựa trên thời gian sẵn có và Độ ưu tiên tương đối (Relative priority) . Lập trình viên chia nhỏ các câu chuyện này thành các Nhiệm vụ phát triển (Development tasks) .

Phát hành nhỏ (Small releases)	Tập hợp các chức năng hữu ích tối thiểu mang lại giá trị kinh doanh được phát triển trước. Việc phát hành diễn ra thường xuyên và bổ sung dần chức năng.
Thiết kế đơn giản (Simple design)	Chỉ thực hiện thiết kế vừa đủ để đáp ứng các yêu cầu hiện tại, không làm thừa.
Phát triển hướng kiểm thử (Test-first development)	Sử dụng một Khung kiểm thử tự động (Automated unit test framework) để viết kiểm thử trước khi cài đặt chức năng mới.
Tái cấu trúc (Refactoring)	Lập trình viên liên tục cải thiện mã nguồn ngay khi tìm thấy điểm có thể tối ưu nhằm giữ mã nguồn đơn giản và dễ bảo trì.
Lập trình cặp (Pair programming)	Các lập trình viên làm việc theo cặp, kiểm tra công việc của nhau và hỗ trợ nhau để hoàn thành tốt công việc.
Sở hữu tập thể (Collective ownership)	Các cặp lập trình viên làm việc trên mọi khu vực của hệ thống, giúp mọi người đều có trách nhiệm với toàn bộ mã nguồn. Bất kỳ ai cũng có thể thay đổi bất cứ thứ gì.
Tích hợp liên tục (Continuous integration)	Ngay khi xong một nhiệm vụ, nó sẽ được tích hợp vào toàn bộ hệ thống. Mọi kiểm thử đơn vị phải vượt qua sau mỗi lần tích hợp.
Tốc độ bền vững (Sustainable pace)	Không chấp nhận làm thêm giờ (Overtime) quá nhiều vì sẽ làm giảm chất lượng mã và năng suất trung hạn.

Khách hàng tại chỗ (On-site customer)	Đại diện người dùng cuối phải có mặt toàn thời gian để hỗ trợ nhóm XP, đóng vai trò là một thành viên mang lại các yêu cầu hệ thống.
--	--

MỞ RỘNG (EXPANSION)

1. **Phát triển hướng kiểm thử (Test-Driven Development - TDD):** XP là "cha đẻ" của khái niệm TDD. Việc viết kiểm thử trước giúp lập trình viên hiểu cực kỳ rõ yêu cầu trước khi đặt bút viết code, đồng thời tạo ra một "lưới an toàn" để tự tin thay đổi mã nguồn sau này.
2. **Lập trình cặp (Pair Programming):** Dù có vẻ tốn nhân lực (2 người 1 máy), nhưng thực tế nó giúp giảm thiểu lỗi ngay từ đầu (**Review code** tức thì) và là cách cực tốt để đào tạo nhân viên mới thông qua việc truyền đạt kinh nghiệm trực tiếp.
3. **Refactoring và Nợ kỹ thuật (Technical Debt):** Việc tái cấu trúc liên tục giúp ngăn chặn sự tích tụ của "nợ kỹ thuật" — những đoạn code viết vội, khó hiểu khiến hệ thống trì trệ về lâu dài.
4. **Thách thức của Khách hàng tại chỗ (On-site Customer):** Trong thực tế, rất khó để một doanh nghiệp cử một nhân sự giỏi ngồi trực tiếp với đội phần mềm 24/7. Đây thường là điểm khó thực hiện nhất của XP trong thực tế

XP VÀ CÁC NGUYÊN TẮC LINH HOẠT (XP AND AGILE PRINCIPLES)

- **Phát triển tăng dần (Incremental development):** * Được hỗ trợ thông qua các **Bản phát hành hệ thống nhỏ và thường xuyên (Small, frequent system releases)**.
- **Sự tham gia của khách hàng (Customer involvement):** * Nghĩa là có sự **Cam kết của khách hàng toàn thời gian (Full-time customer engagement)** với đội ngũ phát triển.
- **Con người thay vì quy trình (People not process):** * Thông qua **Lập trình cặp (Pair programming)**, **Sở hữu tập thể (Collective ownership)** và một quy trình tránh **Làm việc nhiều giờ liên tục (Long working hours)**.
- **Hỗ trợ thay đổi (Change supported):** * Thông qua các **Bản phát hành hệ thống định kỳ (Regular system releases)**.
- **Duy trì sự đơn giản (Maintaining simplicity):** * Thông qua việc **Tái cấu trúc mã nguồn liên tục (Constant refactoring of code)**.

CÁC THỰC TIỄN XP CÓ TẦM ẢNH HƯỞNG (INFLUENTIAL XP PRACTICES)

- **Lập trình cực hạn (Extreme programming)** tập trung mạnh vào khía cạnh kỹ thuật và không dễ để tích hợp với **Thực hành quản lý (Management practice)** tại hầu hết các tổ chức.
- Hệ quả là, mặc dù **Phát triển linh hoạt (Agile development)** có sử dụng các thực hành từ XP, nhưng phương pháp này như định nghĩa ban đầu lại không được sử dụng rộng rãi.
- **Các thực hành cốt lõi (Key practices):**
 - **Câu chuyện người dùng (User stories)** cho phần **Đặc tả (Specification)**.
 - **Tái cấu trúc (Refactoring)**.
 - **Phát triển hướng kiểm thử (Test-first development)**.
 - **Lập trình cặp (Pair programming)**.

CÂU CHUYỆN NGƯỜI DÙNG CHO CÁC YÊU CẦU (USER STORIES FOR REQUIREMENTS)

- Trong XP, khách hàng hoặc người dùng là một phần của **Đội ngũ XP (XP team)** và chịu trách nhiệm đưa ra các **Quyết định về yêu cầu (Decisions on requirements)**.
- **Yêu cầu người dùng (User requirements)** được thể hiện dưới dạng **Câu chuyện người dùng (User stories)** hoặc **Kịch bản (Scenarios)**.
- Những nội dung này được viết trên các **Thẻ (Cards)** và đội ngũ phát triển sẽ chia nhỏ chúng thành các **Nhiệm vụ triển khai (Implementation tasks)**.
- Các nhiệm vụ này là cơ sở để **Ước tính tiến độ và chi phí (Schedule and cost estimates)**.
- Khách hàng chọn các câu chuyện để đưa vào **Bản phát hành kế tiếp (Next release)** dựa trên **Độ ưu tiên (Priorities)** của họ và các ước tính về tiến độ.

MỞ RỘNG (EXPANSION)

1. **Tại sao XP khó áp dụng nguyên bản?:** XP yêu cầu kỷ luật kỹ thuật cực cao và sự hiện diện 24/7 của khách hàng. Tuy nhiên, các dự án hiện đại thường kết hợp **Scrum** (để quản lý quy trình) với **XP** (để áp dụng các kỹ thuật lập trình

như TDD, Pair Programming, Refactoring). Đây gọi là mô hình Hybrid linh hoạt.

2. **Cấu trúc của một Câu chuyện người dùng (User Story):** Một User Story chuẩn thường được viết theo mẫu:
 - *"As a [role], I want [goal/desire] so that [benefit/value]."* * (Với tư cách là [vai trò], tôi muốn [mục tiêu] để [giá trị nhận được]).
 - Điều này giúp đội ngũ tập trung vào **Giá trị kinh doanh (Business value)** thay vì chỉ nhìn vào tính năng kỹ thuật.
3. **Mô hình 3C trong User Stories:**
 - **Card (Thẻ):** Yêu cầu được viết ngắn gọn trên thẻ.
 - **Conversation (Thảo luận):** Chi tiết yêu cầu được làm rõ qua việc trao đổi trực tiếp giữa Dev và khách hàng.
 - **Confirmation (Xác nhận):** Các tiêu chí chấp nhận (**Acceptance Criteria**) để biết khi nào câu chuyện đó hoàn thành.
4. **Tái cấu trúc (Refactoring) và Chất lượng nội tại:** XP coi Refactoring là việc bắt buộc. Nó không phải là "sửa lỗi" mà là cải thiện cấu trúc bên trong của code mà không làm thay đổi hành vi bên ngoài. Điều này giúp hệ thống không bị "giòn" (dễ gãy/lỗi) khi thêm tính năng mới.

SO SÁNH QUY TRÌNH PHÁT TRIỂN (DEVELOPMENT PROCESS COMPARISON)

Dựa trên hình ảnh so sánh giữa hai phương pháp luận:

- **Phát triển dựa trên kế hoạch (Plan-based development):**
 - **Kỹ nghệ yêu cầu (Requirements engineering):** Quá trình xác định các nhu cầu của khách hàng.
 - **Đặc tả yêu cầu (Requirements specification):** Tạo ra văn bản mô tả chi tiết hệ thống.
 - **Thiết kế và triển khai (Design and implementation):** Thực hiện xây dựng phần mềm dựa trên đặc tả đã có.
 - **Yêu cầu thay đổi yêu cầu (Requirements change requests):** Nếu có thay đổi, quy trình buộc phải quay lại bước **Kỹ nghệ yêu cầu (Requirements engineering)** để điều chỉnh lại toàn bộ chuỗi.
- **Phát triển linh hoạt (Agile development):**
 - Các hoạt động **Kỹ nghệ yêu cầu (Requirements engineering)** và **Thiết kế và triển khai (Design and implementation)** được thực hiện trong một vòng lặp kín và liên tục, **không có bước Đặc tả yêu cầu (Requirements specification) trung gian cố định.**

CHU TRÌNH PHÁT HÀNH TRONG XP (THE XP RELEASE CYCLE)

Quy trình này mô tả cách một nhóm XP vận hành để đưa ra một bản phát hành:

- Chọn câu chuyện người dùng cho đợt phát hành này (Select user stories for this release):** Khách hàng chọn các tính năng ưu tiên.
- Chia nhỏ câu chuyện thành các nhiệm vụ (Break down stories to tasks):** Lập trình viên cụ thể hóa các yêu cầu thành công việc kỹ thuật.
- Lập kế hoạch phát hành (Plan release):** Xác định thời gian và nguồn lực.
- Phát triển / Tích hợp / Kiểm thử phần mềm (Develop / integrate / test software):** Giai đoạn thực thi chính.
- Phát hành phần mềm (Release software):** Bàn giao sản phẩm chạy được cho khách hàng.
- Đánh giá hệ thống (Evaluate system):** Thu thập phản hồi để chuẩn bị cho chu trình tiếp theo.

MỘT "CÂU CHUYỆN NGƯỜI DÙNG" VỀ KÊ ĐƠN THUỐC (A 'PRESCRIBING MEDICATION' STORY)

Prescribing medication

Kate is a doctor who wishes to prescribe medication for a patient attending a clinic. The patient record is already displayed on her computer so she clicks on the medication field and can select 'current medication', 'new medication' or 'formulary'.

If she selects 'current medication', the system asks her to check the dose; If she wants to change the dose, she enters the new dose then confirms the prescription.

If she chooses 'new medication', the system assumes that she knows which medication to prescribe. She types the first few letters of the drug name. The system displays a list of possible drugs starting with these letters. She chooses the required medication and the system responds by asking her to check that the medication selected is correct. She enters the dose then confirms the prescription.

If she chooses 'formulary', the system displays a search box for the approved formulary. She can then search for the drug required. She selects a drug and is asked to check that the medication is correct. She enters the dose then confirms the prescription.

The system always checks that the dose is within the approved range. If it isn't, Kate is asked to change the dose.

After Kate has confirmed the prescription, it will be displayed for checking. She either clicks 'OK' or 'Change'. If she clicks 'OK', the prescription is recorded on the audit database. If she clicks on 'Change', she reenters the 'Prescribing medication' process.

Đây là một ví dụ thực tế về cách viết **Câu chuyện người dùng (User story)** dưới dạng kịch bản:

- **Bối cảnh:** Kate là một bác sĩ muốn kê đơn thuốc cho bệnh nhân tại phòng khám. Hồ sơ bệnh nhân đã hiển thị trên máy tính, cô ấy nhấn vào trường thuốc và có thể chọn: **Thuốc hiện tại (Current medication)**, **Thuốc mới (New medication)** hoặc **Danh mục thuốc (Formulary)**.
- **Các tình huống:**
 - Nếu chọn **Thuốc hiện tại (Current medication)**, hệ thống yêu cầu kiểm tra **Liều lượng (Dose)**; nếu muốn thay đổi, cô ấy nhập liều mới và **Xác nhận đơn thuốc (Confirms the prescription)**.
 - Nếu chọn **Thuốc mới (New medication)**, hệ thống giả định cô ấy đã biết loại thuốc cần kê. Cô ấy nhập vài chữ cái đầu, hệ thống hiển thị danh sách gợi ý. Cô ấy chọn thuốc, hệ thống yêu cầu kiểm tra lại tính chính xác, sau đó nhập liều lượng và xác nhận.
 - Nếu chọn **Danh mục thuốc (Formulary)**, hệ thống hiển thị ô tìm kiếm các loại thuốc đã được phê duyệt. Quy trình xác nhận diễn ra tương tự.
- **Kiểm soát:** Hệ thống luôn kiểm tra liều lượng có nằm trong **Phạm vi cho phép (Approved range)** hay không.

- **Hoàn tất:** Sau khi xác nhận, đơn thuốc hiển thị để kiểm tra lần cuối. Nếu nhấn **Đồng ý (OK)**, đơn thuốc được ghi lại vào **Cơ sở dữ liệu kiểm toán (Audit database)**. Nếu nhấn **Thay đổi (Change)**, quy trình quay lại từ đầu.
-

💡 MỞ RỘNG (EXPANSION)

1. **Tính chất của User Story:** Bạn có thể thấy ví dụ về kê đơn thuốc không hề mang tính kỹ thuật (không nói về database hay code). Nó được viết bằng ngôn ngữ của người dùng (bác sĩ). Điều này giúp **Khách hàng (Customer)** và **Lập trình viên (Developer)** hiểu nhau mà không bị rào cản công nghệ.
 2. **Cơ sở dữ liệu kiểm toán (Audit database):** Đây là một khái niệm quan trọng trong các hệ thống phần mềm y tế hoặc tài chính. Mọi hành động (như kê đơn) đều phải được lưu vết để phục vụ việc tra cứu trách nhiệm sau này, đảm bảo tính an toàn và minh bạch.
 3. **Kiểm thử trong chu trình XP:** Trong hình ảnh chu trình XP, bước **Test software** cực kỳ quan trọng. Trong XP, nếu một bản dựng (build) không vượt qua 100% các bài kiểm thử tự động, nó sẽ không được phép phát hành. Điều này đảm bảo tốc độ nhanh nhưng không làm giảm chất lượng.
 4. **Sự đan xen (Inter-leaved):** Trong Agile, việc thiết kế không dừng lại rồi mới code. Thiết kế được điều chỉnh ngay khi đang code (thông qua Refactoring) để phù hợp với thực tế phát sinh.
-

VÍ DỤ VỀ THẺ NHIỆM VỤ (EXAMPLES OF TASK CARDS)

Dựa trên hình ảnh ví dụ về hệ thống kê đơn thuốc:

- **Nhiệm vụ 1 (Task 1):** Thay đổi liều lượng của thuốc đã kê (**Change dose of prescribed drug**).
 - **Nhiệm vụ 2 (Task 2):** Lựa chọn danh mục thuốc (**Formulary selection**).
 - **Nhiệm vụ 3 (Task 3):** Kiểm tra liều lượng (**Dose checking**).
 - Kiểm tra liều lượng là một biện pháp an toàn để đảm bảo bác sĩ không kê liều quá thấp hoặc quá cao gây nguy hiểm.
 - Sử dụng mã định danh danh mục (**Formulary id**) để tra cứu liều lượng tối đa và tối thiểu được khuyến nghị.
 - Nếu nằm ngoài phạm vi, hệ thống đưa ra **Thông báo lỗi (Error message)**. Nếu nằm trong phạm vi, kích hoạt nút **Xác nhận (Confirm)**.
-

TÁI CẤU TRÚC (REFACTORING)

Quan niệm truyền thống (Conventional wisdom): Thiết kế để thay đổi (*design for change*), dự đoán trước các thay đổi $\Rightarrow$ giúp giảm chi phí về sau trong Vòng đời (*Life cycle*).

Lập trình cực hạn (XP): Thực hiện **Cải thiện mã nguồn liên tục (Constant code improvement/refactoring)** để giúp việc thay đổi trở nên dễ dàng hơn khi chúng buộc phải được triển khai.

- Thực hiện các cải tiến phần mềm có thể có ngay cả khi chưa có nhu cầu cấp thiết ngay lập tức.
- **Ưu điểm của Tái cấu trúc (Refactoring advantages):**
 - Cải thiện **Khả năng dễ hiểu (Understandability)** của phần mềm $\Rightarrow$ giảm bớt nhu cầu về **Tài liệu hóa (Documentation)**.
 - Mã nguồn được cấu trúc tốt và rõ ràng $\Rightarrow$ các thay đổi dễ thực hiện hơn.
- **Nhược điểm (Disadvantages):**
 - Một số thay đổi yêu cầu **Tái cấu trúc kiến trúc (Architecture refactoring)** $\Rightarrow$ gây tốn kém chi phí.

Ví dụ về Tái cấu trúc (Examples of Refactoring)

- **Tổ chức lại (Re-organization)** một **Cấu trúc phân cấp lớp (Class hierarchy)** để loại bỏ **Mã nguồn trùng lặp (Duplicate code)**.
- Sắp xếp lại và đổi tên các **Thuộc tính (Attributes)** và **Phương thức (Methods)** để làm chúng dễ hiểu hơn.
- Thay thế **Mã nguồn trực tiếp (Inline code)** bằng các lời gọi tới các **Phương thức (Methods)** đã có sẵn trong **Thư viện chương trình (Program library)**.

PHÁT TRIỂN HƯỚNG KIỂM THỬ (TEST-FIRST DEVELOPMENT)

- Kiểm thử là trọng tâm của XP và XP đã phát triển một phương pháp mà chương trình được kiểm thử sau mỗi lần thay đổi.
- **Các đặc điểm kiểm thử trong XP (XP testing features):**
 - **Phát triển hướng kiểm thử (Test-first development).**
 - **Phát triển kiểm thử tăng dần (Incremental test development)** từ các **Kịch bản (Scenarios)**.
 - **Sự tham gia của khách hàng (User involvement)** trong việc phát triển kiểm thử và **Thẩm định (Validation)**.

- Các **Khung kiểm thử tự động (Automated test harnesses)** được sử dụng để chạy tất cả các **Kiểm thử thành phần (Component tests)** mỗi khi một **Bản phát hành mới (New release)** được xây dựng.
-

Phát triển dựa trên kiểm thử (Test-driven development - TDD)

- Viết kiểm thử trước khi viết mã nguồn giúp làm rõ các **Yêu cầu (Requirements)** cần triển khai.
 - Các bài kiểm thử được viết dưới dạng chương trình thay vì dữ liệu để có thể **Thực thi tự động (Executed automatically)**.
 - Bài kiểm thử bao gồm một bước kiểm tra xem nó đã thực thi chính xác hay chưa.
 - Thường dựa trên một **Khung kiểm thử (Testing framework)** như JUnit.
 - Tất cả các bài kiểm thử cũ và mới đều được chạy tự động khi có **Chức năng mới (New functionality)** được thêm vào, từ đó kiểm tra xem chức năng mới có gây ra lỗi không.
-

SỰ THAM GIA CỦA KHÁCH HÀNG TRONG KIỂM THỬ (CUSTOMER INVOLVEMENT IN TESTING)

- Vai trò của khách hàng là giúp phát triển các **Kiểm thử chấp nhận (Acceptance tests)** cho các **Câu chuyện (Stories)** sẽ được triển khai trong bản phát hành tiếp theo.
 - Khách hàng (một phần của đội ngũ) viết các bài kiểm thử khi quá trình phát triển tiến hành. Do đó, tất cả mã mới đều được **Thẩm định (Validated)** để đảm bảo đúng nhu cầu của khách hàng.
 - **Thách thức:** Những người đóng vai trò khách hàng có thời gian hạn chế và không thể làm việc toàn thời gian với đội phát triển. Họ có thể cảm thấy việc cung cấp yêu cầu là đã đủ đóng góp và có thể **Ngại ngần (Reluctant)** tham gia vào quy trình kiểm thử.
-

MỞ RỘNG (EXPANSION)

1. **Hồi quy (Regression Testing) trong TDD:** Khi slide nói về việc "chạy lại các bài kiểm thử cũ", đó chính là khái niệm **Kiểm thử hồi quy**. Trong Agile, vì mã

nguồn thay đổi liên tục, việc có bộ test tự động giúp đảm bảo rằng khi bạn sửa một chỗ này không làm "hỏng" một chỗ khác đã chạy tốt trước đó.

2. **Mối quan hệ giữa Refactoring và Unit Test:** Bạn không thể thực hiện **Tái cấu trúc (Refactoring)** an toàn nếu không có **Kiểm thử đơn vị (Unit Test)**. Bộ test đóng vai trò chứng minh rằng sau khi bạn "dọn dẹp" code, hành vi của phần mềm vẫn không thay đổi.
 3. **Khung kiểm thử xUnit:** JUnit (cho Java) là ví dụ điển hình nhất, nhưng hầu hết các ngôn ngữ đều có khung tương tự: **PyTest/Unittest** (Python), **NUnit** (.NET), **Jest** (JavaScript).
 4. **Kiểm thử chấp nhận (Acceptance Testing):** Đây là mức kiểm thử cao nhất để trả lời câu hỏi: "Sản phẩm này đã đủ điều kiện để khách hàng trả tiền và đưa vào sử dụng chưa?". Trong Scrum, việc này thường diễn ra trong buổi **Sprint Review**.
-

TỰ ĐỘNG HÓA KIỂM THỬ (TEST AUTOMATION)

- **Tự động hóa kiểm thử (Test automation)** có nghĩa là các bài kiểm thử được viết dưới dạng các **Thành phần có thể thực thi (Executable components)** trước khi **Nhiệm vụ (Task)** được triển khai.
 - Những thành phần kiểm thử này nên mang tính **Độc lập (Stand-alone)**, nên **Mô phỏng (Simulate)** việc gửi dữ liệu đầu vào cần kiểm thử và nên kiểm tra xem kết quả có đáp ứng **Đặc tả đầu ra (Output specification)** hay không.
 - Một **Khung kiểm thử tự động (Automated test framework)** (ví dụ: **JUnit**) là một hệ thống giúp việc viết các bài kiểm thử có thể thực thi và gửi một tập hợp các bài kiểm thử để thực thi trở nên dễ dàng.
- Vì việc kiểm thử được tự động hóa, luôn có một bộ các bài kiểm thử có thể được thực thi một cách nhanh chóng và dễ dàng.
 - Bất cứ khi nào có bất kỳ **Chức năng (Functionality)** nào được thêm vào hệ thống, các bài kiểm thử có thể được chạy và những vấn đề mà mã nguồn mới gây ra có thể được **Phát hiện ngay lập tức (Caught immediately)**.

Ví dụ về Tự động hóa kiểm thử:

TEST CASE DESCRIPTION FOR DOSE CHECKING

Test 4: Dose checking
Input: 1. A number in mg representing a single dose of the drug. 2. A number representing the number of single doses per day.
Tests: 1. Test for inputs where the single dose is correct but the frequency is too high. 2. Test for inputs where the single dose is too high and too low. 3. Test for inputs where the single dose * frequency is too high and too low. 4. Test for inputs where single dose * frequency is in the permitted range.
Output: OK or error message indicating that the dose is outside the safe range.

Hãy tưởng tượng bạn đang code chức năng **Kiểm tra liều lượng thuốc (Dose checking)** như trong hình ảnh bạn đã gửi.

- **Cách thủ công:** Mỗi lần sửa code, bạn phải mở ứng dụng, nhập tên thuốc, nhập liều lượng, bấm nút "Xác nhận" và nhìn bằng mắt xem nó hiện thông báo gì. Mất khoảng 1-2 phút cho mỗi lần thử.
- **Cách tự động (JUnit):** Bạn viết một đoạn mã nhỏ (script) ra lệnh cho máy tính tự nhập 100 trường hợp liều lượng khác nhau (quá thấp, quá cao, vừa đủ, số âm, ký tự lạ...) chỉ trong 1 giây. Mỗi khi bạn thay đổi logic tính toán, bạn chỉ cần bấm "Run", máy tính sẽ báo ngay: "99/100 trường hợp đúng, trường hợp số 50 bị lỗi".

NHỮNG VẤN ĐỀ VỚI PHÁT TRIỂN HƯỚNG KIỂM THỬ TRƯỚC (PROBLEMS WITH TEST-FIRST DEVELOPMENT)

- Các lập trình viên thường thích lập trình hơn là kiểm thử và đôi khi họ đi **Lối tắt (Short cuts)** khi viết các bài kiểm thử.
 - Ví dụ, họ có thể viết các bài **Kiểm thử không đầy đủ (Incomplete tests)**, không kiểm tra hết tất cả các **Ngoại lệ (Exceptions)** có thể xảy ra.
- Một số bài kiểm thử có thể rất khó để viết theo kiểu **Tăng dần (Incrementally)**.
 - Ví dụ, trong một **Giao diện người dùng phức tạp (Complex user interface)**, thường rất khó để viết **Kiểm thử đơn vị (Unit tests)** cho mã nguồn triển khai **Logic hiển thị (Display logic)** và **Luồng công việc (Workflow)** giữa các màn hình.
- Rất khó để đánh giá **Tính đầy đủ (Completeness)** của một bộ bài kiểm thử.

- Mặc dù bạn có thể có rất nhiều bài kiểm thử hệ thống, nhưng bộ kiểm thử của bạn có thể không cung cấp **Độ bao phủ đầy đủ (Complete coverage)**.

Ví dụ về khó khăn trong kiểm thử giao diện (UI): Bạn đang xây dựng giao diện cho ứng dụng quản lý tour du lịch hoặc hệ thống xe buýt thông minh bằng **React** và **TypeScript**.

- Việc viết code để máy tính tự kiểm tra xem "Nút 'Xác nhận' có đổi thành màu đỏ khi bác sĩ nhập sai liều lượng không?" hay "Cửa sổ thông báo có hiện ra đúng giữa màn hình không?" là rất khó khăn và tốn thời gian. Lập trình viên thường có xu hướng bỏ qua phần này và tự kiểm tra bằng mắt cho nhanh, dẫn đến việc thiếu sót các bài test tự động cho giao diện.

MỞ RỘNG (EXPANSION)

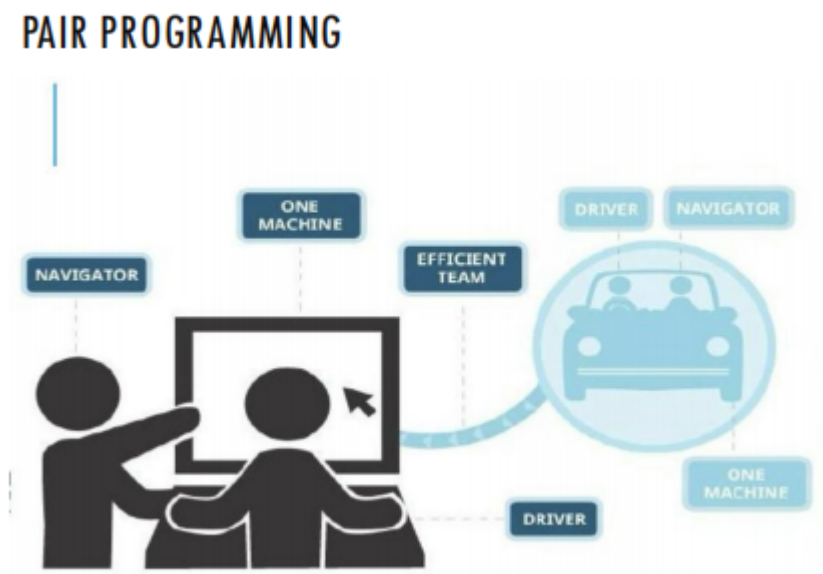
- **Kiểm thử hồi quy (Regression Testing):** Đây là mục tiêu quan trọng nhất của **Tự động hóa kiểm thử**. Trong các dự án bạn đã thực hiện như Hệ thống quản lý tour hay Website bán hàng, khi bạn thêm tính năng "Thanh toán", bạn rất dễ làm hỏng tính năng "Giỏ hàng" đã viết trước đó. Bộ test tự động giúp bạn chắc chắn rằng "tính năng mới không làm hỏng tính năng cũ".
- **Độ bao phủ mã nguồn (Code Coverage):** Đây là chỉ số đo lường xem bao nhiêu phần trăm mã nguồn của bạn đã được các bài kiểm thử chạy qua. Ví dụ: Code của bạn có 10 dòng, nhưng các bài test của bạn chỉ mới chạy qua 6 dòng, vậy **Coverage** là **60%**. Một bộ test có nhiều bài chưa chắc đã tốt nếu chúng chỉ tập trung test đi test lại một đoạn code.
- **Tư duy "Test-first" vs "Test-last":**
 - **Test-last (Truyền thống):** Viết code xong mới viết test. Lúc này bạn thường có tâm lý "code chạy rồi, viết test làm gì nữa cho mệt" dẫn đến việc viết test sơ sài.
 - **Test-first (Agile/XP):** Viết test để hiểu rõ yêu cầu. Nếu bạn không viết được test, nghĩa là bạn chưa hiểu rõ chức năng đó cần làm gì.

LẬP TRÌNH CẶP (PAIR PROGRAMMING)

- **Lập trình cặp (Pair programming):** Làm việc theo cặp, cùng nhau phát triển mã nguồn.

- Đóng vai trò như một **Quy trình xem xét không chính thức (Informal review process)**: vì mỗi dòng mã đều được xem xét bởi nhiều hơn một người.
- Khuyến khích việc **Tái cấu trúc (Refactoring)**: vì toàn bộ đội ngũ đều có thể hưởng lợi từ việc cải thiện mã nguồn hệ thống.
- Các cặp được tạo ra một cách **Động (Dynamically)**: để tất cả các thành viên trong nhóm làm việc với nhau trong suốt **Quy trình phát triển (Development process)**.
- **Chia sẻ kiến thức (Sharing of knowledge)** trong lập trình cặp: Rất quan trọng vì nó giúp giảm thiểu các **Rủi ro tổng thể (Overall risks)** cho dự án khi các thành viên rời nhóm.
- Có một số bằng chứng cho thấy một cặp làm việc cùng nhau mang lại **Hiệu quả (Efficient)** hơn so với hai lập trình viên làm việc riêng lẻ.

Ví dụ về Lập trình cặp:



Hãy tưởng tượng bạn và một người bạn cùng giải một bài toán lập trình khó. Bạn là người cầm bàn phím (**Driver - Người lái**), tập trung viết từng dòng lệnh. Bạn của bạn ngồi bên cạnh (**Navigator - Người điều hướng**) sẽ không nhìn vào cú pháp mà nhìn vào logic tổng thể: "Liệu thuật toán này có xử lý được trường hợp danh sách rỗng không?" hoặc "Cách này chạy nhanh nhưng tốn bộ nhớ quá, mình có cách nào tối ưu hơn không?". Nhờ vậy, code viết ra ít lỗi hơn hẳn so với việc bạn tự làm một mình.

QUẢN TRỊ DỰ ÁN LINH HOẠT (AGILE PROJECT MANAGEMENT)

- Công việc của các **Nhà quản lý dự án phần mềm (Software project managers)** là quản lý dự án sao cho phần mềm được bàn giao **Đúng hạn (On time)** và nằm trong **Ngân sách dự kiến (Planned budget)**.
 - **Tiếp cận tiêu chuẩn (Standard approach) = Kế hoạch dự án (Project plan):**
 - Những gì nên được bàn giao.
 - Khi nào chúng nên được bàn giao.
 - Và ai sẽ làm việc để tạo ra các **Sản phẩm bàn giao (Deliverables)** của dự án.
 - **Quản trị dự án linh hoạt (Agile project management)?** (Đây là một cách tiếp cận khác biệt, tập trung vào sự thích nghi thay vì bám cứng vào kế hoạch ban đầu).
-

SCRUM

- Scrum là một **phương pháp linh hoạt** tập trung vào việc quản lý **Phát triển lặp (Iterative development)** thay vì các thực hành kỹ thuật cụ thể.
 - Có **ba giai đoạn** trong Scrum:
 1. **Giai đoạn khởi đầu (Initial phase):** Là giai đoạn **Lập kế hoạch phác thảo (Outline planning)**, nơi bạn thiết lập các mục tiêu chung cho dự án và thiết kế **Kiến trúc phần mềm (Software architecture)**.
 2. Tiếp theo là một chuỗi các **Chu kỳ nước rút (Sprint cycles)**: Mỗi chu kỳ phát triển một **Phần tăng trưởng (Increment)** của hệ thống.
 3. **Giai đoạn kết thúc dự án (Project closure phase):** Gói gọn dự án, hoàn tất các tài liệu yêu cầu như **Khung trợ giúp hệ thống (System help frames)**, **Hướng dẫn sử dụng (User manuals)** và đánh giá các **Bài học kinh nghiệm (Lessons learned)**.
-



MỞ RỘNG (EXPANSION)

1. **Tại sao Pair Programming lại hiệu quả?:** Nghiên cứu cho thấy dù tốn 2 người cho 1 máy, nhưng số lượng lỗi (bugs) giảm xuống đáng kể (khoảng 15-50%). Chi phí để sửa lỗi sau khi phần mềm đã phát hành đắt hơn nhiều so với việc trả lương cho 2 người ngồi cùng nhau để viết code đúng ngay từ đầu.
2. **Sự khác biệt trong Quản lý (Management Shift):**
 - Trong mô hình truyền thống (Waterfall), người quản lý là "Chỉ huy" (đưa ra mệnh lệnh).

- Trong Agile/Scrum, người quản lý (thường là **Scrum Master**) đóng vai trò là "Người phục vụ" (Servant Leader), tập trung vào việc tháo gỡ khó khăn cho đội ngũ để họ làm việc tốt nhất.
3. **Sprint (Nước rút)**: Một Sprint thường kéo dài từ 2 đến 4 tuần. Điểm mấu chốt là sau mỗi Sprint, bạn **phải** có một phần mềm thực sự chạy được để trình diễn cho khách hàng, chứ không chỉ là các bản báo cáo hay tài liệu giấy.
 4. **Kiến trúc trong Scrum**: Lưu ý slide đề cập đến việc thiết kế kiến trúc ở giai đoạn khởi đầu. Trong Agile, kiến trúc này không cần hoàn hảo 100% ngay lúc đầu (vì nó sẽ tiến hóa), nhưng phải đủ tốt để đội ngũ có thể bắt đầu xây dựng các tính năng đầu tiên.

Thuật ngữ Scrum (Phần A) - SCRUM TERMINOLOGY (A)

Thuật ngữ Scrum (Scrum term)	Định nghĩa (Definition)
Đội ngũ phát triển (Development team)	Một nhóm Tự tổ chức (Self-organizing) gồm các lập trình viên, thường không quá 7 người. Họ chịu trách nhiệm phát triển phần mềm và các Tài liệu dự án thiết yếu (Essential project documents) khác.
Phần tăng trưởng sản phẩm có khả năng bàn giao (Potentially shippable product increment)	Phần tăng trưởng phần mềm được bàn giao sau mỗi Giai đoạn nước rút (Sprint) . Ý tưởng là phần này phải ở trạng thái "hoàn tất" và không cần làm thêm việc gì khác (như kiểm thử) để tích hợp vào sản phẩm cuối cùng. Trong thực tế, điều này không phải lúc nào cũng đạt được.
Danh mục sản phẩm (Product backlog)	Đây là danh sách các hạng mục "cần làm" mà nhóm Scrum phải giải quyết. Chúng có thể là các định nghĩa tính năng, Yêu cầu phần mềm (Software requirements) , Câu chuyện người dùng (User stories) hoặc mô tả các nhiệm vụ bổ sung cần thiết (như xác định kiến trúc hoặc tài liệu người dùng).

Chủ sở hữu sản phẩm (Product owner)	Một cá nhân (hoặc một nhóm nhỏ) có nhiệm vụ xác định các tính năng hoặc yêu cầu sản phẩm, Ưu tiên (Prioritize) chúng và liên tục xem xét Danh mục sản phẩm (Product backlog) để đảm bảo dự án đáp ứng các nhu cầu kinh doanh quan trọng. Họ có thể là khách hàng hoặc người quản lý sản phẩm.
--	---

3. Thuật ngữ Scrum (Phần B) - SCRUM TERMINOLOGY (B)

Thuật ngữ Scrum (Scrum term)	Định nghĩa (Definition)
Họp Scrum hàng ngày (Daily Scrum)	Một cuộc họp hàng ngày của nhóm Scrum để xem xét tiến độ và ưu tiên công việc cần làm trong ngày đó. Lý tưởng nhất, đây nên là một cuộc họp ngắn trực tiếp với đầy đủ thành viên.
Người điều phối Scrum (ScrumMaster)	Chịu trách nhiệm đảm bảo quy trình Scrum được tuân thủ và hướng dẫn nhóm sử dụng Scrum hiệu quả. Họ đóng vai trò kết nối với phần còn lại của công ty và đảm bảo nhóm không bị xao nhãng bởi các Sự can thiệp bên ngoài (Outside interference) .
Giai đoạn nước rút (Sprint)	Một Vòng lặp phát triển (Development iteration) . Các Sprint thường kéo dài từ 2 đến 4 tuần.
Tốc độ (Velocity)	Một ước tính về lượng công việc trong Danh mục sản phẩm (Product backlog) mà nhóm có thể hoàn thành trong một Sprint duy nhất. Hiểu được tốc độ của nhóm giúp họ ước tính khối lượng công việc và là cơ sở để đo lường việc cải thiện hiệu suất.

💡 MỞ RỘNG (EXPANSION) - VÍ DỤ MINH HỌA

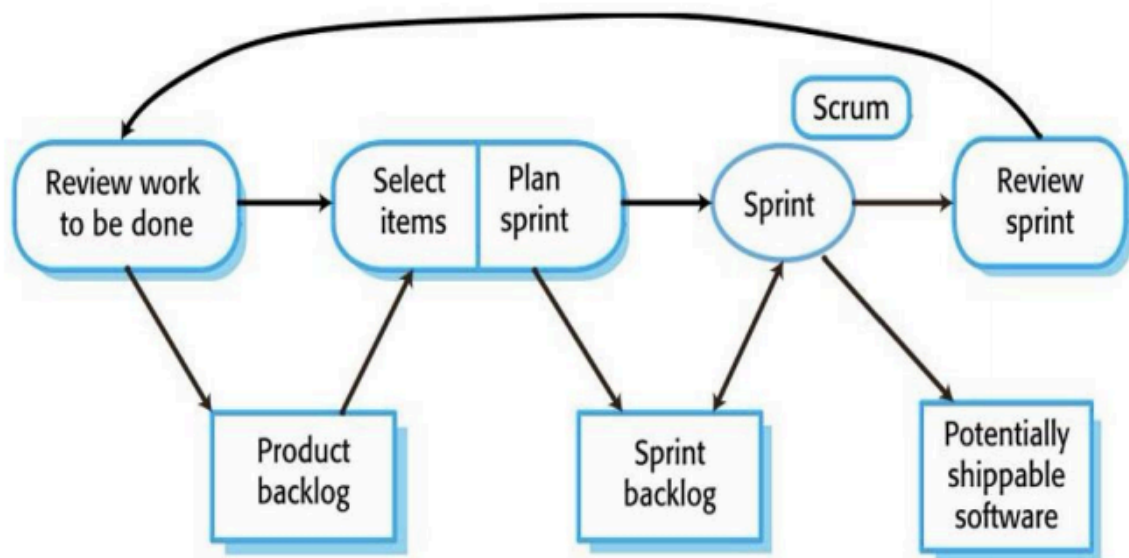
Để các thuật ngữ trên không còn "bay bổng", hãy liên hệ với một ví dụ thực tế: **Xây dựng ứng dụng "Smart Bus" (Xe buýt thông minh)** mà bạn đang quan tâm.

1. **Product Backlog:** Là danh sách tất cả tính năng bạn muốn: "Xem bản đồ", "Nạp tiền vé", "Thông báo xe đến", "Chat với lái xe".
2. **Product Owner:** Là người quyết định: "Sáng nay hành khách cần Xem bản đồ nhất, hãy làm cái đó trước!".
3. **Sprint:** Cả nhóm quyết định trong 2 tuần tới chỉ tập trung làm đúng tính năng "Xem bản đồ".
4. **ScrumMaster:** Nếu sếp của công ty chạy vào bảo: "Này, làm thêm cái tính năng Đặt vé máy bay đi!", ScrumMaster sẽ nói: "Không, nhóm đang làm Sprint cho Xe buýt, đừng làm phiền họ!".
5. **Daily Scrum:** Mỗi sáng, 7 người đứng quanh cái bảng, mỗi người nói 3 ý: "Hôm qua tôi làm xong cái icon xe buýt", "Hôm nay tôi code phần định vị GPS", "Tôi đang bị kẹt ở chỗ bản đồ chưa hiện đúng tên đường".
6. **Velocity:** Sau 2 tuần, nhóm làm được 5 tính năng nhỏ. Vậy "Tốc độ" của nhóm là 5. Lần sau PO đưa 10 tính năng, nhóm sẽ biết ngay là "quá tải" không làm nổi.

Lưu ý quan trọng cho kỳ thi: ScrumMaster **không phải** là Project Manager (Quản lý dự án). Project Manager thường ra lệnh "Anh làm cái này, chị làm cái kia". ScrumMaster thì hỏi "Anh chị gặp khó khăn gì để tôi giúp?" và đảm bảo mọi người chơi đúng luật Scrum.

CHU KỲ NƯỚC RÚT TRONG SCRUM (SCRUM SPRINT CYCLE)

Sơ đồ mô tả quy trình quản lý dự án theo khung làm việc Scrum:



Giai đoạn nước rút (Sprint): Có độ dài cố định, thông thường từ **2-4 tuần**.

Danh mục sản phẩm (Product backlog): Danh sách tất cả các công việc cần thực hiện cho dự án.

- **Quy trình chọn lọc (The selection):** Lựa chọn các tính năng từ **Danh mục sản phẩm (Product backlog)** để phát triển trong Sprint. Nhóm tự tổ chức để thực hiện công việc này.
- **Trong giai đoạn "Nước rút" (During "sprint" stage):**
 - Nhóm làm việc biệt lập với khách hàng và tổ chức để tập trung cao độ.
 - Mọi giao tiếp bên ngoài đều thông qua **Người điều phối Scrum (Scrum Master)** để bảo vệ nhóm khỏi sự xao nhãng.
- **Kết thúc giai đoạn nước rút (End of the sprint):** Công việc được xem xét và trình diễn cho **Các bên liên quan (Stakeholders)**. Kết quả là một **Phần mềm có khả năng bàn giao tiềm năng (Potentially shippable software)**.

💡 MỞ RỘNG (EXPANSION)

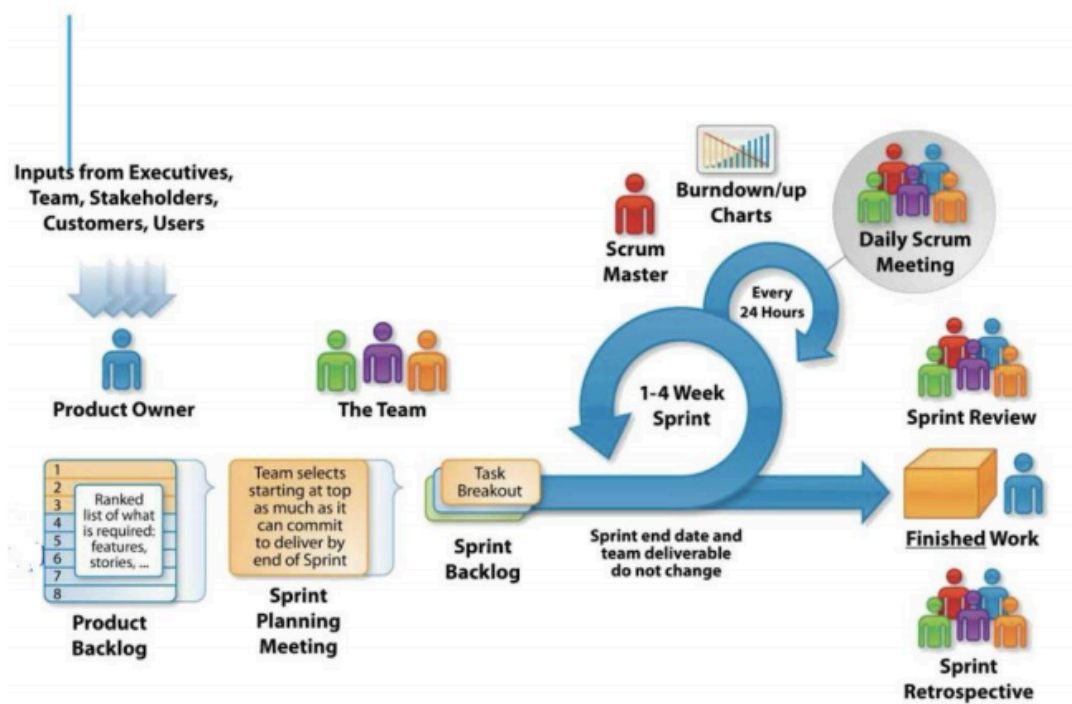
1. **Tính biệt lập trong Sprint (Isolation):** Tại sao nhóm cần biệt lập? Trong Agile, thay đổi là tốt nhưng thay đổi **ngay giữa chừng** một Sprint 2 tuần sẽ làm rối loạn kế hoạch. Scrum Master đóng vai trò như "người gác cổng" để đảm bảo nhóm hoàn thành đúng cam kết trong Sprint đó.
2. **Định nghĩa về "Hoàn thành" (Definition of Done - DoD):** Slide nhắc đến **Potentially shippable product increment**. Để đạt được điều này, nhóm phải

thống nhất một DoD, ví dụ: Code đã viết, đã Review, đã chạy Unit Test 100% đạt, và đã cập nhật tài liệu hướng dẫn. Nếu thiếu một bước, nó chưa được gọi là "Hoàn thành".

3. **Tầm quan trọng của Audit Database trong ví dụ y tế:** Trong các hệ thống đặc thù (Y tế, Tài chính), các bài kiểm thử không chỉ tập trung vào tính năng (Bác sĩ kê được đơn) mà còn phải kiểm tra tính toàn vẹn của dữ liệu kiểm toán (Mọi thao tác thay đổi liều lượng có được lưu lại chính xác để truy xuất trách nhiệm không?).

SCRUM SPRINT CYCLE – ANOTHER VIEW

<https://www.youtube.com/watch?v=XJeaPyWOCaw>

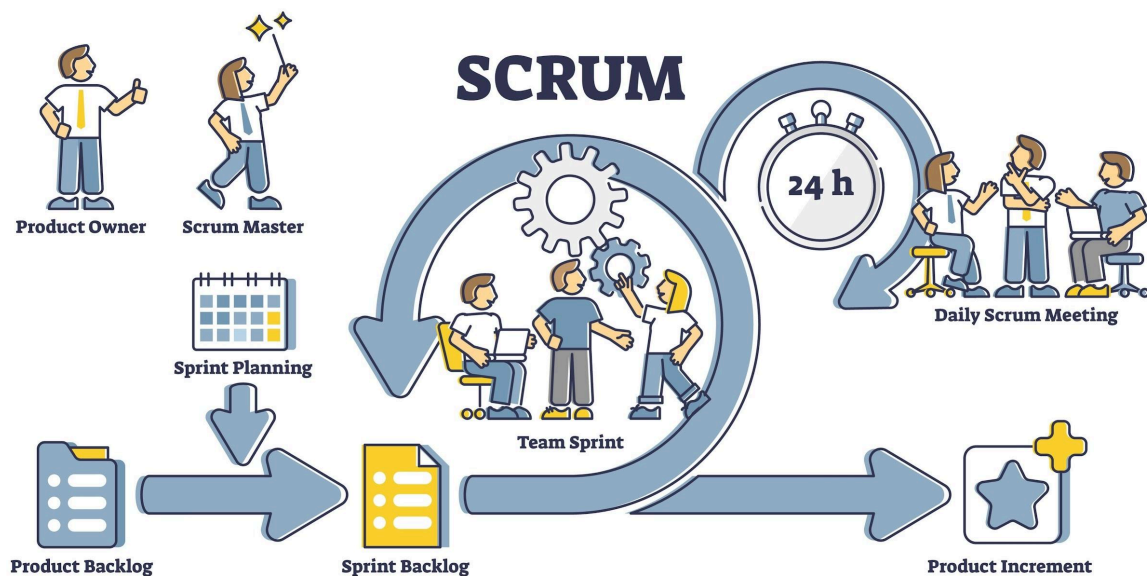


<https://www.youtube.com/watch?v=XJeaPyWOCaw>

LÀM VIỆC NHÓM TRONG SCRUM (TEAMWORK IN SCRUM)

- **"Người điều phối Scrum" (Scrum master):** Là một Người hỗ trợ/Điều phối (Facilitator):
 - Sắp xếp các **Cuộc họp hàng ngày (Daily meetings)** và theo dõi **Danh mục công việc (Backlog)** cần thực hiện.

- Ghi lại các quyết định và đo lường **Tiến độ (Progress)** dựa trên danh mục công việc.
- Giao tiếp với khách hàng và **Ban quản lý (Management)** bên ngoài nhóm.
- **Các cuộc họp hàng ngày ngắn (Scrums)** cho tất cả thành viên trong nhóm:
 - Chia sẻ thông tin.
 - Mô tả **Tiến độ (Progress)** kể từ cuộc họp trước.
 - Những **Vấn đề (Problems)** đã nảy sinh.
 - Những gì được lập kế hoạch cho ngày tiếp theo ⇒ **Lập lại kế hoạch (Re-plan)** nếu cần thiết.



LỢI ÍCH CỦA SCRUM (SCRUM BENEFITS)

- Sản phẩm được chia nhỏ thành một tập hợp các **Phần nhỏ dễ quản lý và dễ hiểu (Manageable and understandable chunks)**.
- Các **Yêu cầu không ổn định (Unstable requirements)** không làm trì trệ tiến độ.
- Toàn bộ nhóm đều có **Khả năng quan sát/Sự minh bạch (Visibility)** về mọi thứ, do đó **Giao tiếp trong nhóm (Team communication)** được cải thiện.
- Khách hàng thấy được việc **Bàn giao đúng hạn (On-time delivery)** của các phần tăng trưởng và nhận được **Phản hồi (Feedback)** về cách sản phẩm hoạt động.
- **Sự tin tưởng (Trust)** giữa khách hàng và các lập trình viên được thiết lập, tạo ra một **Văn hóa tích cực (Positive culture)** mà ở đó mọi người đều kỳ vọng dự án sẽ thành công.

TỔNG KẾT (SUMMARY)

- Các phương pháp linh hoạt (Agile methods) là các phương pháp **Phát triển tăng dần (Incremental development)** tập trung vào:
 - Phát triển phần mềm nhanh (Rapid software development).
 - Các Bản phát hành phần mềm thường xuyên (Frequent releases of the software).
 - Giảm thiểu Chi phí gián tiếp của quy trình (Process overheads) bằng cách tối giản Tài liệu hóa (Documentation).
 - Và tạo ra Mã nguồn chất lượng cao (High-quality code).
- Các thực hành phát triển linh hoạt (Agile development practices) bao gồm:
 - Câu chuyện người dùng (User stories) cho việc Đặc tả hệ thống (System specification).
 - Các bản phát hành phần mềm thường xuyên.
 - Cải tiến phần mềm liên tục (Continuous software improvement).
 - Phát triển hướng kiểm thử (Test-first development).
 - Sự tham gia của khách hàng (Customer participation) trong đội ngũ phát triển.

-
- Scrum là một phương pháp linh hoạt cung cấp một **Khung quản lý dự án (Project management framework)**.
 - Nó tập trung xoay quanh một tập hợp các **Giai đoạn nước rút (Sprints)**, vốn là các khoảng thời gian cố định khi một **Phần tăng trưởng hệ thống (System increment)** được phát triển.
 - Nhiều phương pháp phát triển thực tế là sự kết hợp giữa **Phát triển dựa trên kế hoạch (Plan-based development)** và **Phát triển linh hoạt (Agile development)**.

MỞ RỘNG (EXPANSION)

1. **Ba câu hỏi trong Daily Scrum:** Để cụ thể hóa phần "mô tả tiến độ", mỗi thành viên thường trả lời 3 câu hỏi:
 - *Hôm qua tôi đã làm được gì?*
 - *Hôm nay tôi sẽ làm gì?*
 - *Tôi có đang gặp khó khăn/trở ngại (Impediments) nào không?*

2. **Ví dụ:** Trong dự án "Smart Bus", một lập trình viên có thể nói: "Hôm qua tôi xong phần API định vị, hôm nay tôi nhúng bản đồ vào giao diện, tôi đang bị kẹt vì GPS giả lập trên máy ảo chạy rất chậm".
3. **Tại sao lại là "Người hỗ trợ" (Facilitator) mà không phải "Quản lý" (Manager)?:** Scrum Master không giao việc cho từng người. Thay vào đó, họ dọn dẹp các "chướng ngại vật" (ví dụ: máy tính hỏng, thiếu server test) để nhóm tự làm việc hiệu quả nhất.
4. **Mô hình hỗn hợp (Hybrid Model):** Slide cuối nhắc đến việc kết hợp hai phương pháp. Trong thực tế, các doanh nghiệp lớn thường dùng **Waterfall (Thác nước)** ở cấp cao để lập kế hoạch ngân sách hàng năm, nhưng dùng **Scrum** ở cấp đội ngũ lập trình để thực hiện dự án. Việc này giúp cân bằng giữa "Sự ổn định của tổ chức" và "Sự linh hoạt của kỹ thuật".
5. **Sự minh bạch (Visibility):** Trong Scrum, người ta thường dùng **Bảng Kanban (Kanban Board)** hoặc **Biểu đồ Burndown (Burndown Chart)** để ai đi ngang qua cũng biết nhóm đang làm tới đâu, còn bao nhiêu việc, có bị chậm tiến độ hay không mà không cần đọc báo cáo dài dòng.

Thẻ 2

Để giúp bạn thoát khỏi sự mơ hồ và hình dung rõ ràng hơn về cách vận hành của **Phát triển linh hoạt (Agile development)** và **Lập trình cực hạn (Extreme Programming - XP)**, tôi sẽ cụ thể hóa các khái niệm bằng những ví dụ thực tế dưới đây.

1. Ví dụ về Câu chuyện người dùng (User Story) và Thẻ nhiệm vụ (Task Card)

Giả sử bạn đang xây dựng một ứng dụng ngân hàng. Thay vì viết một bản đặc tả kỹ thuật dày 50 trang, bạn bắt đầu bằng những mẫu tin nhỏ trên thẻ giấy.

- **Câu chuyện người dùng (User Story):** "Với tư cách là khách hàng, tôi muốn xem lịch sử giao dịch trong 30 ngày qua để tôi có thể quản lý chi tiêu của mình."
 - **Chia nhỏ thành các Nhiệm vụ (Tasks):** Các lập trình viên sẽ lấy thẻ này và chia nó ra thành các đầu việc nhỏ trên **Thẻ nhiệm vụ (Task cards)**:
 - **Nhiệm vụ 1:** Thiết kế giao diện danh sách giao dịch trên điện thoại.
 - **Nhiệm vụ 2:** Viết câu lệnh truy vấn cơ sở dữ liệu để lấy dữ liệu giao dịch theo ID khách hàng.
 - **Nhiệm vụ 3:** Xử lý định dạng ngày tháng và tiền tệ hiển thị.
-

2. Ví dụ về Phát triển hướng kiểm thử (Test-Driven Development - TDD)

Hãy tưởng tượng bạn đang viết chức năng "Kiểm tra mật khẩu" (mật khẩu phải có ít nhất 8 ký tự).

1. **Viết kiểm thử trước (Test-first):** Bạn viết một đoạn mã kiểm thử yêu cầu: "Nếu nhập '123', hệ thống phải báo lỗi". Lúc này, vì bạn chưa viết code chức năng, bài kiểm thử này sẽ **Thất bại (Fail)**.
 2. **Viết mã nguồn (Write code):** Bạn chỉ viết đúng một dòng code: `if len(password) < 8: return Error.`
 3. **Chạy lại kiểm thử:** Bây giờ bài kiểm thử sẽ **Vượt qua (Pass)**.
 4. **Tái cấu trúc (Refactoring):** Bạn xem lại code, nếu thấy cách viết chưa tối ưu hoặc tên biến chưa rõ ràng, bạn sửa lại ngay mà không sợ làm hỏng chức năng vì đã có bộ kiểm thử canh giữ.
-

3. Ví dụ về Lập trình cặp (Pair Programming)

Trong một văn phòng theo phong cách XP:

- **Người lái (Driver):** Bạn A ngồi gõ mã nguồn cho chức năng thanh toán. Bạn A tập trung vào cú pháp và logic chi tiết.
 - **Người điều hướng (Navigator):** Bạn B ngồi cạnh quan sát, không gõ phím. Bạn B sẽ nhìn bức tranh lớn hơn: "Này, nếu khách hàng nhấn thanh toán hai lần liên tiếp thì sao?", hoặc "Chỗ này mình nên dùng thư viện xử lý tiền tệ thay vì số thập phân thông thường để tránh sai số".
 - **Kết quả:** Lỗi được phát hiện ngay lập tức, và kiến thức về hệ thống được chia sẻ cho cả hai người (tránh tình trạng chỉ một người biết code chỗ đó).
-

4. Ví dụ về Tái cấu trúc (Refactoring)

Giả sử code của bạn có một đoạn logic tính thuế được lặp đi lặp lại ở 5 nơi khác nhau (trong trang giỏ hàng, trang thanh toán, trang hóa đơn, v.v.).

- **Vấn đề:** Khi thuế thay đổi, bạn phải đi tìm 5 nơi để sửa. Rất dễ sót lỗi.
 - **Thực hiện Tái cấu trúc (Refactoring):** Bạn gom toàn bộ logic đó vào một **Phương thức (Method)** duy nhất mang tên `calculateTax()`. Bây giờ, 5 nơi kia chỉ cần gọi phương thức này.
 - **Lợi ích:** Mã nguồn sạch hơn, dễ hiểu hơn và giảm bớt nhu cầu phải viết **Tài liệu (Documentation)** giải thích cho từng nơi.
-

5. Sự khác biệt thực tế giữa Agile và Thác nước (Waterfall)

Hãy so sánh việc xây dựng một chiếc xe hơi:

- **Thác nước (Plan-driven):** Bạn dành 6 tháng thiết kế bản vẽ, 6 tháng làm khung gầm, 6 tháng làm động cơ. Sau 2 năm khách hàng mới thấy chiếc xe. Nếu lúc đó họ bảo "Tôi muốn xe điện chứ không phải xe xăng", bạn phải đập đi làm lại từ đầu.
- **Linh hoạt (Agile):**
 - **Vòng lặp 1:** Bạn đưa khách hàng một cái ván trượt (họ đã có thể di chuyển được ngay).
 - **Vòng lặp 2:** Bạn gắn thêm tay lái thành xe scooter.
 - **Vòng lặp 3:** Bạn nâng cấp thành xe đạp.
 - **Kết quả:** Khách hàng có **Phần mềm đang chạy tốt (Working software)** để dùng ngay và góp ý từ sớm.

💡 MỞ RỘNG (EXPANSION)

- **Nợ kỹ thuật (Technical Debt):** Nếu bạn bỏ qua **Tái cấu trúc (Refactoring)** để chạy theo tiến độ, bạn đang vay một khoản "nợ kỹ thuật". Theo thời gian, code của bạn sẽ trở nên "giòn" và dễ vỡ (chính chỗ này hỏng chỗ kia), khiến việc phát triển chậm lại đáng kể.
- **Thay đổi tư duy (Mental Shift):** Agile không chỉ là một bộ quy tắc, nó là một tư duy. Thay vì sợ hãi **Thay đổi yêu cầu (Requirements change)**, Agile coi đó là cơ hội để tạo ra sản phẩm thực sự hữu ích cho khách hàng.
- **Tại sao XP lại "Cực hạn" (Extreme)?:** Nó được gọi là cực hạn vì nó đẩy các thực hành tốt nhất lên mức cao nhất. Nếu kiểm thử là tốt, chúng ta sẽ kiểm thử mọi lúc (TDD). Nếu xem xét mã nguồn là tốt, chúng ta sẽ xem xét liên tục (Pair programming).

1. Scrum và Agile có quan hệ gì?

Hãy dùng hình ảnh về "**Lối sống lành mạnh**":

- **Agile (Linh hoạt) là Triết lý/Tư duy:** Nó giống như mục tiêu "Tôi muốn sống lành mạnh". Để đạt được điều đó, bạn có các nguyên tắc như: ăn nhiều rau, tập thể dục thường xuyên, ngủ đủ giấc. Nhưng nó không bắt bạn chính xác 8 giờ sáng phải chạy bao nhiêu km.
- **Scrum là một Khung làm việc (Framework) cụ thể:** Nó giống như một "Chế độ tập luyện" cụ thể để thực hiện triết lý trên. Nó quy định rõ: Sáng thứ 2 tập Gym, chiều thứ 3 bơi lội, mỗi bữa ăn phải có bao nhiêu gram protein.

Nói cách khác: Agile là "đích đến" và "nguyên tắc đạo đức", còn Scrum là "công cụ" và "quy trình" để đưa bạn đến đó. Bạn có thể làm Agile mà không dùng Scrum (dùng XP, Kanban...), nhưng nếu bạn làm đúng Scrum, bạn đang thực hiện Agile.

1 Scrum là gì? (hiểu nhanh)

👉 **Scrum giúp team làm phần mềm từng phần nhỏ, liên tục cải tiến**, thay vì làm một lần thật to rồi mới giao sản phẩm.

- Chia dự án thành các **chu kỳ ngắn** (gọi là **Sprint**)
- Mỗi Sprint thường kéo dài **1–4 tuần**

- Cuối mỗi Sprint:
 - ✓ Có sản phẩm chạy được
 - ✓ Có đánh giá & rút kinh nghiệm
-

2 Scrum giải quyết vấn đề gì?

Trong phát triển phần mềm:

- Yêu cầu khách hàng **hay thay đổi**
- Làm lâu mới xong → dễ **sai hướng**
- Team khó phối hợp

👉 Scrum giúp:

- Thích nghi nhanh với thay đổi
 - Giao sản phẩm sớm
 - Tăng minh bạch & teamwork
-

3 vai trò chính trong Scrum (rất hay ra đề thi 📖)

♦ 1. Product Owner (PO)

- Đại diện cho **khách hàng / người dùng**
- Quyết định:
 - Làm tính năng gì?
 - Ưu tiên cái nào trước?

- Quản lý **Product Backlog**

👉 Ví dụ: quyết định *web bán hàng cần giỏ hàng trước hay thanh toán trước*

♦ 2. Scrum Master

- **Người hướng dẫn Scrum**
- Không phải sếp ❌
- Giúp team:
 - Hiểu đúng Scrum
 - Gỡ khó khăn
 - Làm việc hiệu quả

👉 Giống như “trọng tài + huấn luyện viên”

♦ 3. Development Team

- Dev, tester, designer...
 - **Tự tổ chức**, không bị chỉ đạo vi mô
 - Trực tiếp tạo ra **Increment (phần sản phẩm hoàn chỉnh)**
-

📖 4 Các sự kiện (Events) trong Scrum

🔄 Sprint

- Chu kỳ làm việc (1–4 tuần)

- Không thay đổi mục tiêu giữa Sprint
-

Sprint Planning

- Lập kế hoạch cho Sprint
 - Trả lời:
 - Sprint này làm gì?
 - Làm như thế nào?
-

Daily Scrum (Daily Standup)

- **Họp 15 phút mỗi ngày**
 - Trả lời 3 câu:
 1. Hôm qua làm gì?
 2. Hôm nay làm gì?
 3. Có vướng gì không?
-

Sprint Review

- Demo sản phẩm cho PO / khách hàng
 - Nhận feedback
-

Sprint Retrospective

- Nhìn lại cách làm việc
 - Cải thiện quy trình cho Sprint sau
-

5 Các khái niệm quan trọng cần nhớ

- **Product Backlog:** Danh sách toàn bộ yêu cầu
 - **Sprint Backlog:** Những việc chọn làm trong Sprint
 - **Increment:** Phiên bản sản phẩm có thể bàn giao
 - **User Story:** Mô tả yêu cầu theo góc nhìn người dùng
👉 “Là người dùng, tôi muốn..., để...”
-

6 Scrum khác gì Waterfall? (so sánh nhanh)

Waterfall	Scrum
Làm tuần tự	Làm lặp
Khó thay đổi	Thích nghi nhanh
Giao sản phẩm cuối	Giao liên tục
Ít feedback	Feedback thường xuyên

7 Ví dụ thực tế (gắn với đề án của bạn)

Đề tài: **Website thương mại điện tử**

Sprint 1:

- Đăng ký / đăng nhập
- Quản lý tài khoản

Sprint 2:

- Danh sách sản phẩm
- Giỏ hàng

Sprint 3:

- Thanh toán
- Quản lý đơn hàng

👉 Mỗi Sprint xong là **web chạy được thêm tính năng**